Java Full Stack Development

3. Query Optimization

Cost Based Optimizer

CBO

- The optimizer
  - Takes a SQL statement and generates a query plan
  - A query plan is a tree of operations that retrieves and combines data
  - The optimizer evaluates multiple candidate plans and selects the lowest-cost one
- How cost is estimated
  - The optimizer uses statistics about tables and indexes
  - Statistics include
    - Row counts, column cardinality, value distribution (histograms), block counts
    - Statistics are gathered by DBMS_STATS.GATHER_TABLE_STATS or automatically by Oracle
  - Stale statistics lead to poor plan choices

Input	Description
Table statistics	Row count, block count, average row length
Column statistics	Number of distinct values, min/max, null count
Histograms	Value distribution for skewed columns
Index statistics	Depth, leaf blocks, clustering factor
System statistics	CPU speed, I/O throughput

Execution Plan

- SQL statements are declarative
  - Describe what data is wanted
- The execution plan
  - Is a specific, concrete sequence of physical operations
  - Steps Oracle has decided to perform in order to produce the correct result
  - Before Oracle executes a single query, the optimizer generates this plan
  - Essentially an algorithm that the execution engine then carries out step by step

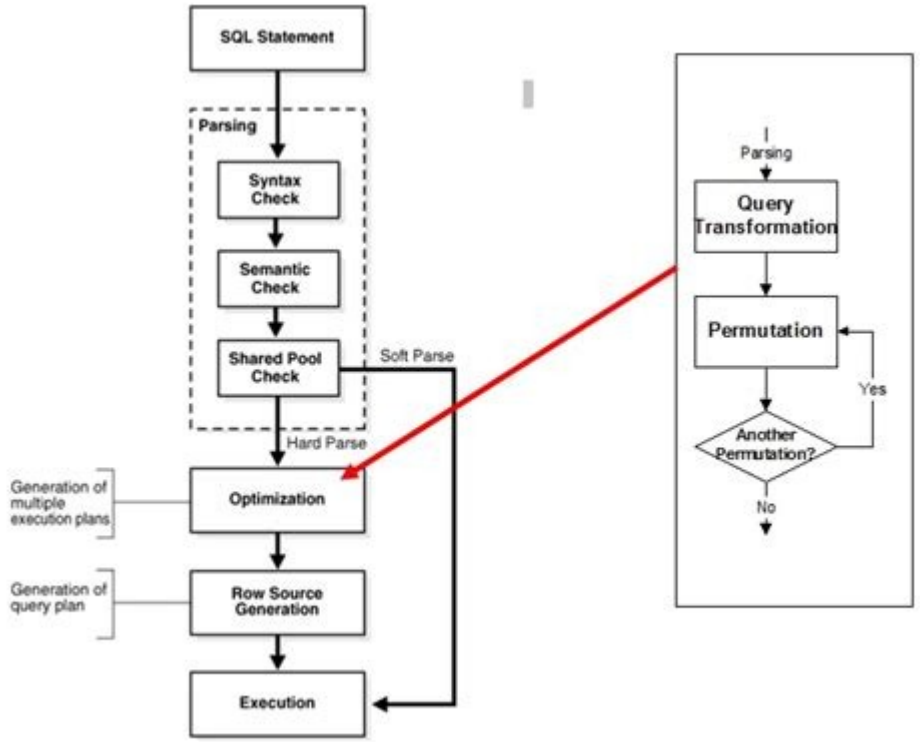

Image Credit: http://logicalread.com/oracle-hard-parsing-h01/

Execution Plan

- Each step in the plan
  - Represents one physical operation
  - Common examples are
    - Read every block in a table
    - Look up a value in an index, take two sets of rows and find the matching pairs
    - Sort a set of rows, group rows together and compute an aggregate.
  - Each operation takes input
    - Either directly from storage or from the output of a previous operation
  - And produces output that flows into the next operation

```
EXPLAIN PLAN FOR
SELECT c.customer_name, COUNT(o.order_id) AS order_count
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
WHERE  c.status = 'ACTIVE'
GROUP BY c.customer_name;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
-------------------------------------------------------------------------------
| Id | Operation                    | Name           | Rows | Cost (%CPU)|
-------------------------------------------------------------------------------
|  0 | SELECT STATEMENT             |                |   50 |   120    |
|  1 |  HASH GROUP BY               |                |   50 |   120    |
|* 2 |   HASH JOIN                  |                |  500 |   115    |
|* 3 |    TABLE ACCESS FULL         | CUSTOMERS      | 1000 |    45    |
|  4 |    INDEX FAST FULL SCAN      | IDX_ORDERS_CUST |50000 |    60    |
-------------------------------------------------------------------------------
Predicate Information (identified by operation id):
   2 - access("O"."CUSTOMER_ID"="C"."CUSTOMER_ID")
   3 - filter("C"."STATUS"='ACTIVE')
```

Execution Plan

- The operations are arranged hierarchically
  - The deepest operations in the hierarchy are the ones that touch storage directly
    - They read blocks from tables or indexes
  - Their output flows upward to joining operations
    - These combine results from two sources
  - The output of joins flows upward to filtering, sorting, or aggregation operations
  - At the very top is the operation that returns the final result set to the caller

```
EXPLAIN PLAN FOR
SELECT c.customer_name, COUNT(o.order_id) AS order_count
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
WHERE  c.status = 'ACTIVE'
GROUP BY c.customer_name;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
-------------------------------------------------------------------------------
| Id | Operation                    | Name           | Rows | Cost (%CPU)|
-------------------------------------------------------------------------------
|  0 | SELECT STATEMENT             |                |   50 |   120    |
|  1 |  HASH GROUP BY               |                |   50 |   120    |
|* 2 |   HASH JOIN                  |                |  500 |   115    |
|* 3 |    TABLE ACCESS FULL         | CUSTOMERS      | 1000 |    45    |
|  4 |    INDEX FAST FULL SCAN      | IDX_ORDERS_CUST |50000 |    60    |
-------------------------------------------------------------------------------
Predicate Information (identified by operation id):
   2 - access("O"."CUSTOMER_ID"="C"."CUSTOMER_ID")
   3 - filter("C"."STATUS"='ACTIVE')
```

Execution Plan

- There are many ways to execute an SQL statement
  - For example
    - A three-table query could read the tables in six different orders
    - Each table could be accessed by a full scan or by one of several indexes
    - Each join could use a nested loop, a hash join, or a sort-merge join
  - Every combination of these choices produces the same correct answer
    - But with dramatically different performance characteristics
    - The execution plan is Oracle's selection from among all of these valid alternatives that the optimizer calculated to have the lowest cost given the current statistics

```
EXPLAIN PLAN FOR
SELECT c.customer_name, COUNT(o.order_id) AS order_count
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
WHERE  c.status = 'ACTIVE'
GROUP BY c.customer_name;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
-------------------------------------------------------------------------------
| Id | Operation                    | Name           | Rows | Cost (%CPU)|
-------------------------------------------------------------------------------
|  0 | SELECT STATEMENT             |                |   50 |   120    |
|  1 |  HASH GROUP BY               |                |   50 |   120    |
|* 2 |   HASH JOIN                  |                |  500 |   115    |
|* 3 |    TABLE ACCESS FULL         | CUSTOMERS      | 1000 |    45    |
|  4 |    INDEX FAST FULL SCAN      | IDX_ORDERS_CUST |50000 |    60    |
-------------------------------------------------------------------------------
Predicate Information (identified by operation id):
   2 - access("O"."CUSTOMER_ID"="C"."CUSTOMER_ID")
   3 - filter("C"."STATUS"='ACTIVE')
```

Join Types

- Nested Loop Join
  - Performance depends entirely on
    - How many rows are in the outer table
    - How quickly Oracle can find matching rows in the inner table
  - If the inner table has an index on the join column, each lookup is fast
    - One index traversal plus one or a few table block reads
  - If there is no index on the inner table's join column
    - Oracle must scan the entire inner table once for every row in the outer table
    - Catastrophic at any meaningful scale.

```
For each row in the outer table:
    Find all matching rows in the inner table
    Emit the combined result
```

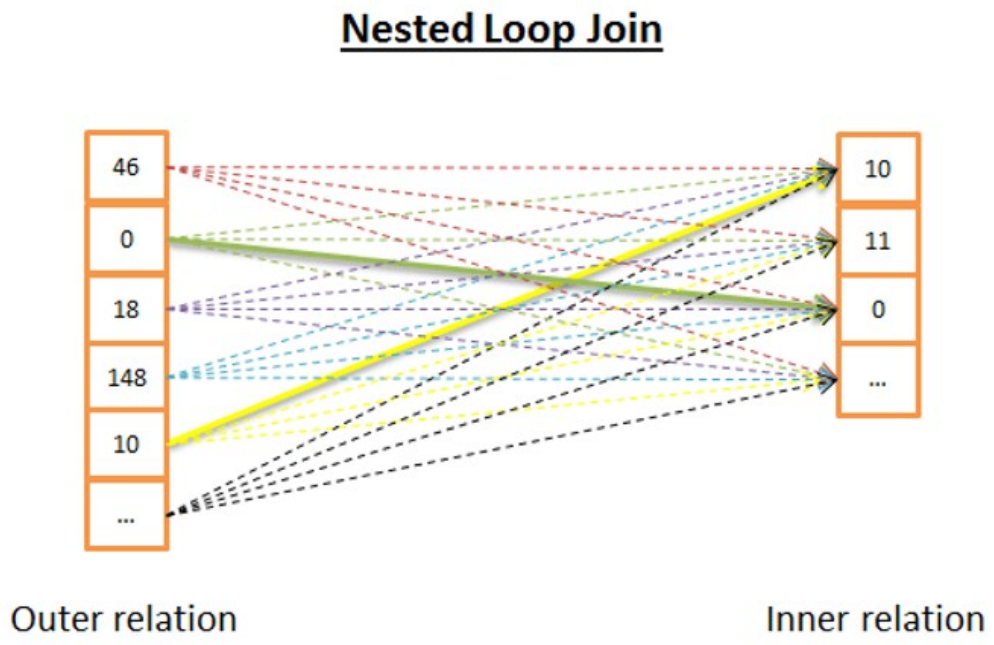

Image Credit: https://dbtut.com/index.php/2018/06/23/join-types-in-sql-server-execution-plan/

Join Types

- Nested Loop Join
  - Right choice when
    - The outer result set is small, a few rows to a few hundred rows
    - And the inner table has a good index on the join column
  - Wrong choice when the outer result set is large
    - Because the cost multiplies directly with the number of outer rows

```
For each row in the outer table:
    Find all matching rows in the inner table
    Emit the combined result
```

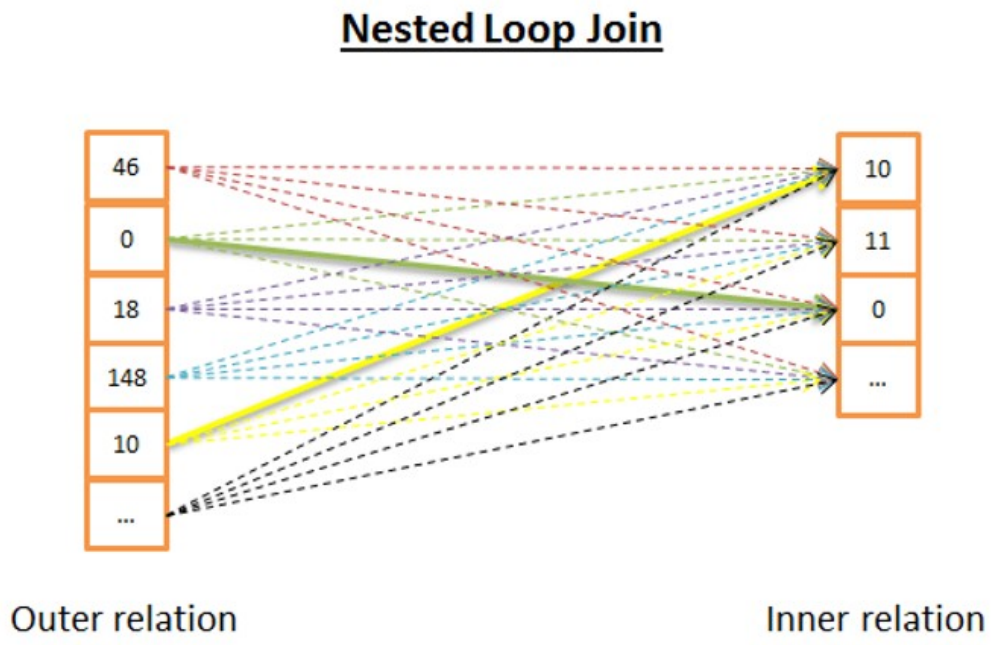

Image Credit: https://dbtut.com/index.php/2018/06/23/join-types-in-sql-server-execution-plan/

Join Types

- Hash Join
  - Oracle reads one of the two tables entirely
  - Builds an in-memory hash table from its rows using the join column as the hash key
  - Then reads the second table row by row and probes the hash table to find matches
- Critical requirement
  - The build-side hash table fits in the memory allocated to the session

```
Phase 1 -- Build:
    Read the smaller table entirely
    Build a hash table in memory keyed on the join column

Phase 2 -- Probe:
    Read the larger table row by row
    For each row, compute the hash of its join column
    Look up that hash in the hash table
    Emit any matches found
```

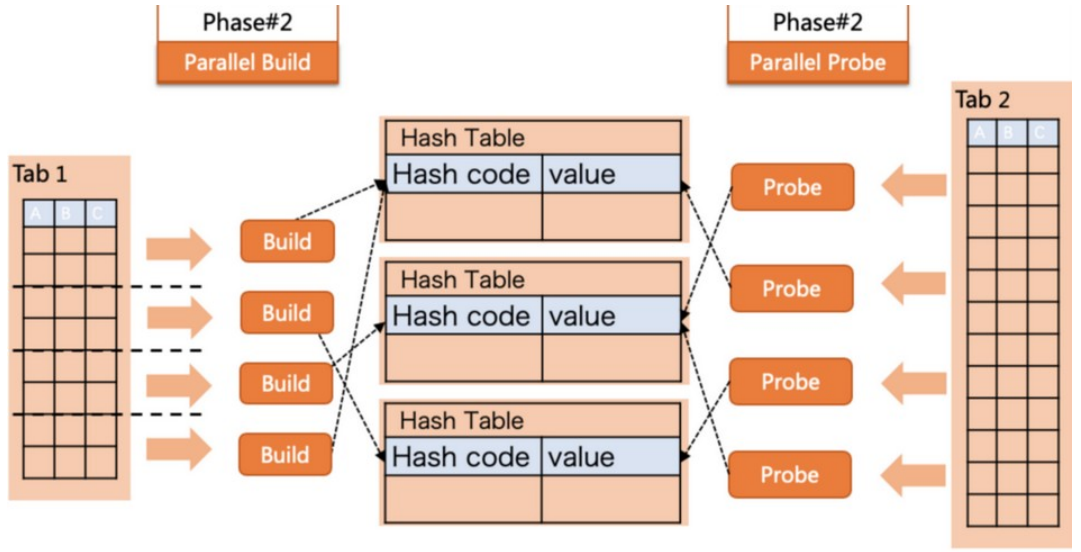

Image Credit: https://help.aliyun.com/document_detail/178156.html

Join Types

- Hash Join
  - If the table fits in memory
    - The hash join requires exactly one full read of each table
    - Extremely efficient for large joins where no useful index exists
  - If it does not fit
    - Oracle spills the overflow to temporary disk storage and performs multiple passes
    - This degrades performance significantly

```
Phase 1 -- Build:
    Read the smaller table entirely
    Build a hash table in memory keyed on the join column

Phase 2 -- Probe:
    Read the larger table row by row
    For each row, compute the hash of its join column
    Look up that hash in the hash table
    Emit any matches found
```

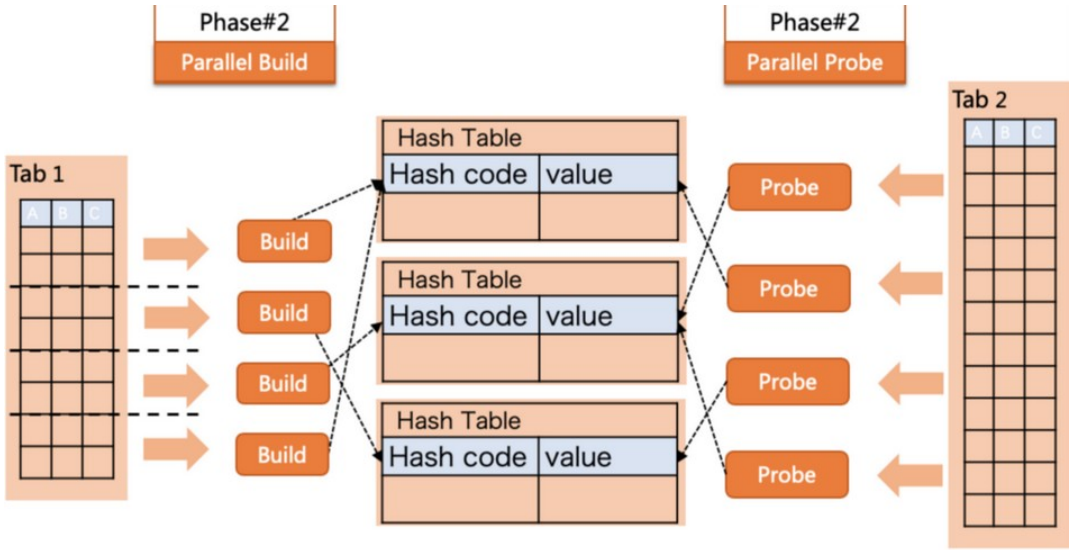

Image Credit: https://help.aliyun.com/document_detail/178156.html

Join Types

- Hash Join
  - Optimizer's default choice for joining large tables on equality conditions where an index-based nested loop is not viable
  - Do not require any index on the join columns
  - They only work for equality joins
    - Cannot hash join on a range condition like WHERE a.date > b.date.

```
Phase 1 -- Build:
    Read the smaller table entirely
    Build a hash table in memory keyed on the join column

Phase 2 -- Probe:
    Read the larger table row by row
    For each row, compute the hash of its join column
    Look up that hash in the hash table
    Emit any matches found
```

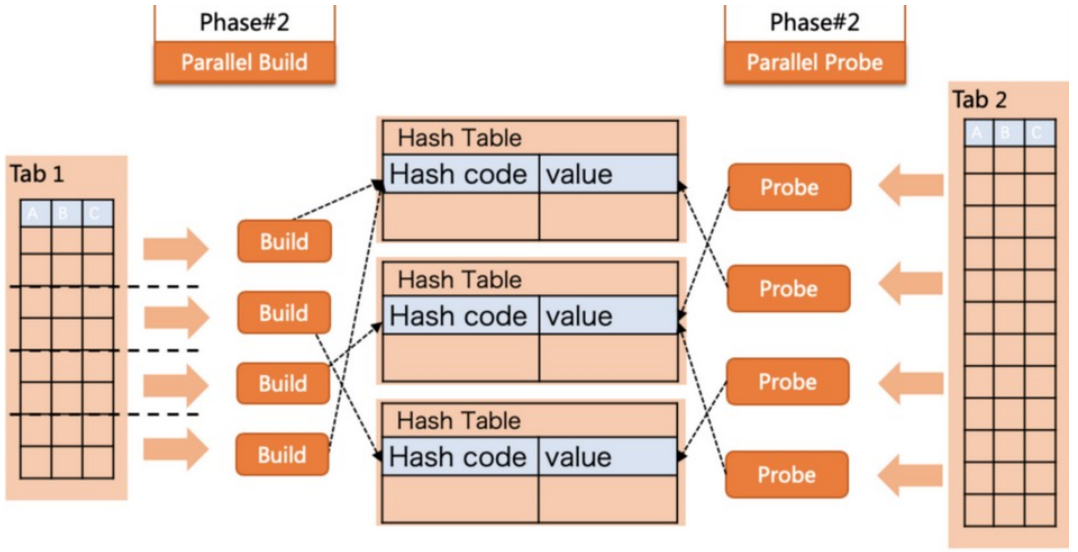

Image Credit: https://help.aliyun.com/document_detail/178156.html

Join Types

- Sort-Merge Join
  - Works by sorting both tables on the join column
  - Then walking through both sorted results simultaneously merging rows that share the same join key value
  - The merge phase is extremely efficient
    - Reads through each sorted result exactly once in sequence
  - The expensive part is the sorting
  - Requires reading all rows and writing them back in order
    - Usually involving temporary disk space for large inputs

```
Phase 1 -- Sort:
    Sort the first table by the join column
    Sort the second table by the join column

Phase 2 -- Merge:
    Walk through both sorted results in parallel
    Where join keys match, emit the combined row
    Advance whichever pointer is behind
```

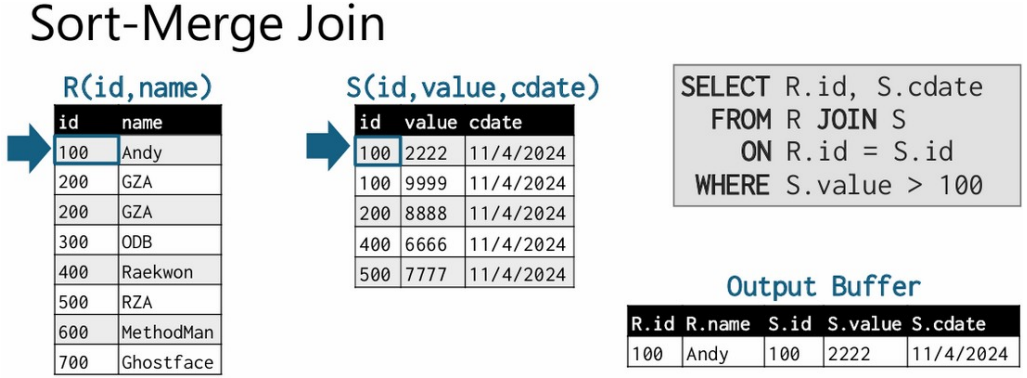

Image Credit: https://blog.csdn.net/m0_74331272/article/details/144234852

Join Types

- Sort-Merge Join
  - Most useful when the inputs are already sorted
  - For example
    - When both tables are accessed via an index range scan on the join column, which returns rows in index order
    - Then Oracle can skip the sort phase entirely and go straight to the merge, making it very efficient
  - Handles non-equality join conditions that hash joins cannot, such as range joins
  - Hash join usually outperforms it for large unsorted inputs
  - Nested loop outperforms it for small inputs

```
Phase 1 -- Sort:
    Sort the first table by the join column
    Sort the second table by the join column

Phase 2 -- Merge:
    Walk through both sorted results in parallel
    Where join keys match, emit the combined row
    Advance whichever pointer is behind
```

Sort-Merge Join

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)

id	value	cdate
100	2222	11/4/2024
100	9999	11/4/2024
200	8888	11/4/2024
400	6666	11/4/2024
500	7777	11/4/2024

```
SELECT R.id, S.cdate
  FROM R JOIN S
    ON R.id = S.id
 WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	11/4/2024

Image Credit: https://blog.csdn.net/m0_74331272/article/details/144234852

Join Types

- How the optimizer chooses
  - Estimates the cost of each applicable algorithm given
    - The estimated row counts
    - Available indexes, available memory
    - The nature of the join condition
  - Then selects the lowest cost option

Situation	Likely choice
Small outer result set, indexed inner join column	Nested loop
Large tables, equality join, no useful index	Hash join
Inputs already sorted on the join column	Sort-merge
Non-equality join condition	Nested loop or sort-merge (hash join not applicable)
Hash table too large for available PGA memory	Sort-merge may be preferred over a spilling hash join

Join Order

- Matters as much as the algorithm
  - For a three-table join
    - Oracle must decide which two tables to join first and then join the result to the third
    - The number of possible orderings grows factorially with the number of tables
    - Three tables have six possible orderings
    - Five tables have one hundred and twenty.
  - Optimizer evaluates the most promising orderings
    - Selects the combination of order and algorithm with the lowest estimated total cost
    - Cardinality estimation errors compound
    - A wrong row count estimate for the first join produces a wrong input estimate for the second join, and so on, leading to a plan that looks locally reasonable at each step but is globally wrong

Situation	Likely choice
Small outer result set, indexed inner join column	Nested loop
Large tables, equality join, no useful index	Hash join
Inputs already sorted on the join column	Sort-merge
Non-equality join condition	Nested loop or sort-merge (hash join not applicable)
Hash table too large for available PGA memory	Sort-merge may be preferred over a spilling hash join

Questions?